

SKILL WEEK-2

1. To Install Linux VM, Configure GCC, Setup Git Repository, Create Project Structure, Understand Shell Architecture, Build Initial Makefile.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
    pid_t pid;

    pid = fork();

    if (pid < 0) {
        printf("Fork failed!\n");
    }
    else if (pid == 0) {
        printf("This is the Child Process.\n");
        printf("Child PID = %d\n", getpid());
    }
    else {
        printf("This is the Parent Process.\n");
        printf("Parent PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
    }

    return 0;
}
```

```
fatal: empty ident name (for <laasya@laasya.localdomain>) not allowed
laasya@laasya:~/oss$ git config --global user.name "laasya"
laasya@laasya:~/oss$ git config --global user.email "laasya02678@gmail.com"
laasya@laasya:~/oss$ git config --global user.name
laasya
laasya@laasya:~/oss$ git config --global user.email
laasya02678@gmail.com
laasya@laasya:~/oss$ git commit -m "Initial project structure"
[master (root-commit) e59ddd1] Initial project structure
17 files changed, 0 insertions(+), 0 deletions(-)
create mode 100644 .gitignore
create mode 100644 LICENSE
create mode 100644 Makefile
create mode 100644 README.md
create mode 100644 docs/design.md
create mode 100644 include/builtin.h
create mode 100644 include/executor.h
create mode 100644 include/parser.h
create mode 100644 include/shell.h
create mode 100644 scripts/run.sh
create mode 100644 src/builtin.c
create mode 100644 src/executor.c
create mode 100644 src/main.c
create mode 100644 src/parser.c
create mode 100644 src/shell.c
create mode 100644 tests/test_executor.c
create mode 100644 tests/test_parser.c
laasya@laasya:~/oss$ nano src/main.c
laasya@laasya:~/oss$ gcc src/main.c -o bin/my_shell
laasya@laasya:~/oss$ ls -l bin
total 16
-rwxr-xr-x 1 laasya laasya 16088 Aug 24 03:58 my_shell
laasya@laasya:~/oss$ ./bin/my_shell
This is the Parent Process.
Parent PID = 15013
Child PID = 15014
This is the Child Process.
Child PID = 15014
laasya@laasya:~/oss$ |
```

```

laasya@laasya:~/oss$ nano src/main.c
laasya@laasya:~/oss$ gcc src/main.c -o bin/my_shell
laasya@laasya:~/oss$ ls -l bin
total 16
-rwxr-xr-x 1 laasya laasya 16088 Aug 24 03:58 my_shell
laasya@laasya:~/oss$ ./bin/my_shell
This is the Parent Process.
Parent PID = 15013
Child PID = 15014
This is the Child Process.
Child PID = 15014
laasya@laasya:~/oss$ pid = fork();
-bash: syntax error near unexpected token '('
laasya@laasya:~/oss$ gcc src/main.c -o bin/my_shell
laasya@laasya:~/oss$ ls -l bin
total 16
-rwxr-xr-x 1 laasya laasya 16088 Aug 24 03:59 my_shell
laasya@laasya:~/oss$ ./bin/my_shell
This is the Parent Process.
Parent PID = 15029
Child PID = 15030
This is the Child Process.
Child PID = 15030
laasya@laasya:~/oss$ |

```

```

#include <stdio.h>
#include <unistd.h>

int main() {
    printf("Before exec()\n");

    execlp("ls", "ls", "-l", NULL);

    printf("After exec()\n");

    return 0;
}

```

2. To Analyze process abstraction, execute `fork()`, understand `exec()` family, analyze parent-child relationships, inspect process tree, practice system call tracing.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    char command[100];

    while (1) {
        printf("my_shell$ ");
        fflush(stdout);

        if (fgets(command, sizeof(command), stdin) == NULL) {
            break;
        }

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0) {
            break;
        }

        pid_t pid = fork();

        if (pid < 0) {
            perror("fork failed");
        }
        else if (pid == 0) {
            execlp(command, command, (char *)NULL);

            perror("exec failed");
            exit(1);
        }
        else {
            wait(NULL);

```

3.

```

    command[strcspn(command, "\n")] = '\0';

    if (strcmp(command, "exit") == 0) {
        break;
    }

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
    }
    else if (pid == 0) {
        execlp(command, command, (char *)NULL);

        perror("exec failed");
        exit(1);
    }
    else {
        wait(NULL);
    }
}

return 0;
}
|

```

^G Help	^O Write Out	^W Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^_\ Replace	^U Paste	^J Justify

```

laasya@laasya:~/oosp$ nano src/main.c
laasya@laasya:~/oosp$ gcc src/main.c -o bin/my_shell
laasya@laasya:~/oosp$ ./bin/my_shell
my_shell$ ls
LICENSE  README.md  bin    include  scripts  tests
Makefile assets    docs  obj      src
my_shell$ date
Mon Aug 24 04:05:33 UTC 2026
my_shell$ exit
laasya@laasya:~/oosp$ gcc src/main.c -o bin/my_shell
./bin/my_shell
my_shell$ |

```